Московский государственный технический университет
имени Н. Э. Баумана

Кафедра _____

ОТЧЕТ

по лабораторным работам № 3-6

по дисциплине «Проектирование сетевых приложений»

Тема: «Разработка клиент-серверного web-приложения на примере CodeSlicer»

Студент: _____

Группа: _____

Преподаватель: _____

Москва, 2026

Введение

Лабораторные работы № 3-6 являются продолжением лабораторных работ № 1-2. На первом этапе были изучены базовые средства HTML, CSS и JavaScript и разработан интерактивный калькулятор в стилистике Kodik. Во второй части курса рассматривается развитие web-приложения: работа с коллекциями данных, Node.js, REST API, AJAX, CORS, Fetch API, промисы и сборка клиентского кода.

В качестве предметной области для лабораторных работ № 3-6 выбран CodeSlicer - local-first инструмент для анализа программных проектов и оценки влияния изменений кода. Реальный CodeSlicer строит граф связей проекта, выполняет impact analysis и review изменений, а также предоставляет несколько интерфейсов: CLI, локальный web-интерфейс Local Hub, расширение VS Code и интеграции с AI-клиентами через skills и MCP [2, 3].

Связь с предыдущим отчетом сохраняется и на уровне экосистемы: CodeSlicer поддерживает работу с Kodik как с одним из AI-клиентов. Поэтому переход от интерфейса в стилистике Kodik к проекту CodeSlicer является содержательно последовательным: сначала рассматривается базовый web-интерфейс, затем - более крупная система для разработчика.

При этом требования учебных лабораторных работ не полностью совпадают с производственной архитектурой CodeSlicer. Например, текущий Local Hub использует same-origin Fetch API и локальный Python backend, тогда как лабораторная работа № 4 требует Node.js и JSON-файл, а лабораторная работа № 5 - XMLHttpRequest и демонстрацию CORS. Поэтому в отчете разделяются два уровня: реальный проект CodeSlicer и его учебный web-модуль управления анализируемыми проектами, который последовательно реализует

требования ЛР3-6. Это позволяет не искажать устройство исходного проекта и одновременно выполнить методические требования курса [1, 2].

Цели и задачи работ

- Изучить Node.js и npm и построить динамический интерфейс на основе коллекции mock-объектов.
- Реализовать REST API с хранением данных в JSON-файле и проверить CRUD-операции.
- Связать frontend и backend через AJAX и XMLHttpRequest и исследовать проблему CORS.
- Перейти к Fetch API, Promise и async/await, выполнить сборку frontend и развернуть его вместе с API.
- На всех этапах сохранять одну предметную область - проекты, передаваемые CodeSlicer для анализа.

1 Общая характеристика проекта CodeSlicer

1.1 Назначение проекта

CodeSlicer предназначен для анализа крупных кодовых баз, в которых изменение одной функции может затронуть несколько модулей, сервисов, API-маршрутов, frontend-компонентов и тестов. Основная проблема, сформулированная авторами проекта, состоит в том, что локально корректное изменение не всегда является корректным для системы целиком: разработчик или AI-агент может видеть отдельный файл, но не полный путь зависимости [3].

CodeSlicer решает эту задачу не обычным поиском по строкам, а построением проверяемого графа проекта. В графе хранятся узлы и связи между ними, а для связи могут сохраняться статус разрешения, confidence, provenance и evidence chain. Благодаря этому система может не только показать, какие

элементы потенциально затронуты изменением, но и объяснить, почему они попали в область влияния [2, 3].

1.2 Основные возможности

Возможность	**Назначение**
Анализ проекта	Определение языков, файлов, модулей, функций, маршрутов, тестов и других сущностей.
Semantic graph	Формирование связей между элементами проекта с сохранением происхождения доказательств.
Impact analysis	Поиск области влияния изменения вверх и вниз по графу.
Review изменений	Анализ diff и выделение обязательных, рекомендуемых и неопределенных областей проверки.
Test selection	Выбор тестов, связанных с измененными узлами и маршрутами.
Local Hub	Локальный браузерный интерфейс для просмотра графа и результатов анализа.
VS Code Cockpit	Работа с анализом и review непосредственно из VS Code.
CLI, skills, MCP	Использование CodeSlicer из терминала и в AI-клиентах.

1.3 Упрощенная архитектура

Общий конвейер CodeSlicer можно представить как последовательное преобразование исходного проекта в набор фактов, затем в семантический граф и далее в результаты impact analysis. В статье о проекте отдельно выделяются inventory, scan plan, extraction, normalization, semantic binding, precision resolution, support packs, quality gates и impact query [3]. Для целей web-отчета эту схему можно укрупнить.

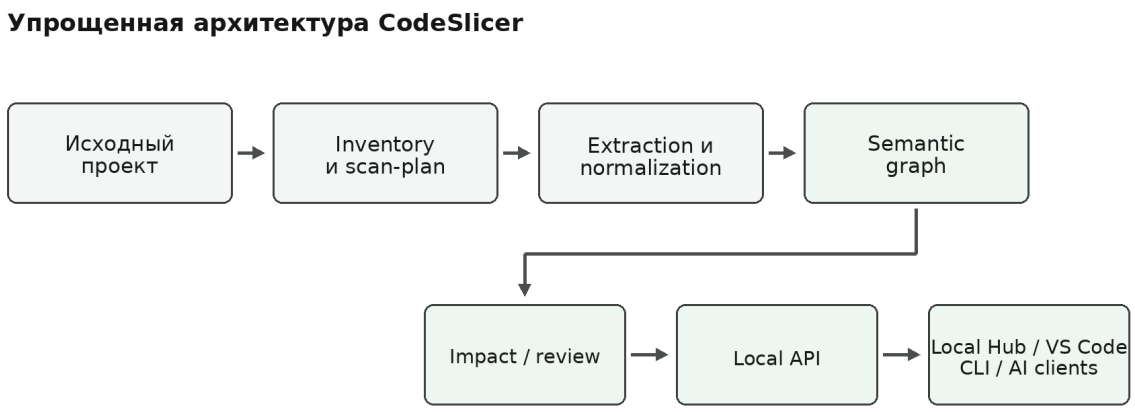


Рисунок 1 - Упрощенная архитектура CodeSlicer

На первом этапе определяется область анализа и состав файлов. Далее extractors извлекают факты из исходного кода. После нормализации и семантического связывания формируется граф. Impact- и review-запросы используют этот граф для ответа на прикладные вопросы. Результаты могут быть показаны через Local Hub, VS Code, CLI или переданы AI-клиенту.

1.4 Интерфейсы и связь с Kodik

В репозитории CodeSlicer предусмотрены несколько способов работы с системой. README рекомендует VS Code Cockpit как короткий путь к первому review; Local Hub предоставляет web-представление большой карты и результатов анализа; CLI предназначен для автоматизации и ручной работы; agent skills и optional MCP дают структурированный workflow для coding agents [2].

В документации отдельно указана поддержка установки skills для Kodik. Это связывает CodeSlicer с тем же типом developer-oriented среды, который использовался как визуальный ориентир в ЛР1-2. CodeSlicer при этом не

заменяет IDE: он предоставляет ей проверяемый контекст проекта и данные об области влияния изменения [2, 3].

1.5 Web-часть реального проекта

В репозитории существует отдельный каталог frontend. Текущий Local Hub представляет собой небольшую SPA с маршрутами #map и #graphify. Согласно frontend/README.md, браузер не содержит fallback-графа или mock-данных и отображает ответы same-origin local API. Для анализа используются GET- и POST-маршруты, например /api/health, /api/state, /api/graph, /api/analyze и /api/graph/projection [2].

Клиентский модуль api-client.js использует fetch() с относительными URL, cache: "no-store" и credentials: "same-origin". Это важно для дальнейшей связи с ЛР6: производственный web-интерфейс CodeSlicer уже соответствует современной схеме same-origin frontend + API. Однако для ЛР3-5 эта схема специально упрощается и реализуется по этапам, требуемым методическими указаниями.

```
async function request(path, options) {
  const response = await fetch(path, {
    cache: 'no-store',
    credentials: 'same-origin',
    ...options
  });
  return await response.json();
}
```

1.6 Учебная адаптация для лабораторных работ

В рамках лабораторных работ предметной сущностью выбран анализируемый проект. Пользовательский сценарий соответствует реальной логике CodeSlicer: пользователь добавляет проект, видит его основные параметры и затем может передать его в анализатор. Учебный модуль не пытается воспроизвести весь semantic engine; его задача - последовательно показать развитие web-архитектуры.

Этап	Учебная реализация	Связь с CodeSlicer
ЛР3	Mock-массив проектов, карточки, фильтр, подробная страница	Модель списка локальных проектов для анализа
ЛР4	Node.js REST API и projects.json	Серверный слой для хранения метаданных проектов
ЛР5	XMLHttpRequest и CORS между двумя origin	Подключение frontend к API
ЛР6	fetch, async/await, bundle и same-origin	Приближение к текущей схеме Local Hub

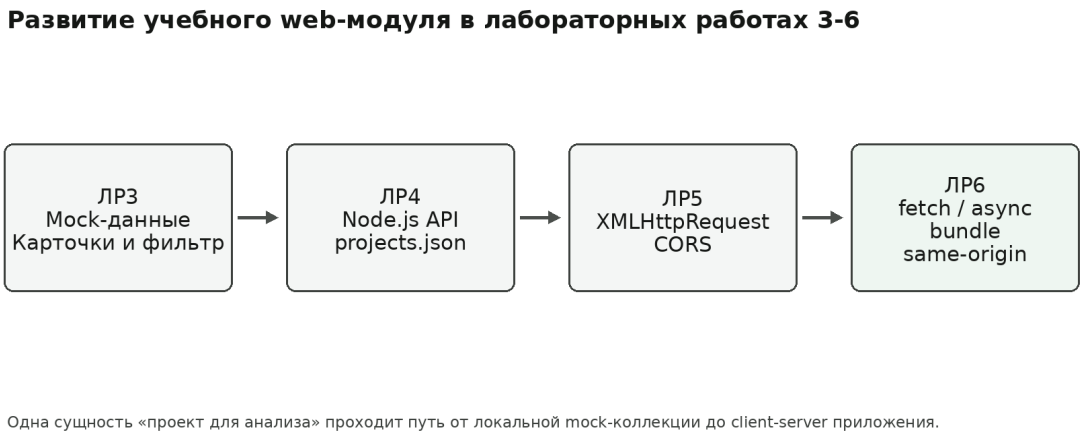


Рисунок 2 - Последовательное развитие учебного web-модуля

2 Лабораторная работа № 3. Node.js, npm и интерфейс с карточками

2.1 Постановка задачи

Согласно методическим требованиям необходимо познакомиться с Node.js и npm, сверстать страницу списка с карточками и фильтрацией, реализовать страницу подробной информации, хранить данные в mock-коллекции, добавить копирование первой карточки и удаление карточек. В header обеих страниц должна присутствовать кнопка «Домой» [1].

2.2 Модель данных

Для предметной области CodeSlicer одна карточка соответствует локальному проекту, который пользователь может анализировать. Минимальная модель содержит идентификатор, название, основной язык, число файлов и статус анализа.

```
const projects = [
  { id: 1, name: 'FastAPI', language: 'Python', files: 3099, status: 'Analyzed' },
  { id: 2, name: 'Express', language: 'JavaScript', files: 202, status: 'Ready' },
  { id: 3, name: 'Gin', language: 'Go', files: 119, status: 'Ready' }
];
```

Названия примеров выбраны не случайно: FastAPI, Gin и Express используются в текущих real-project validation сценариях CodeSlicer. В отчете они выступают только как демонстрационные карточки проектов.

2.3 Структура клиентской части

Файл	**Назначение**
index.html	Страница списка проектов и фильтрации.
project.html	Страница подробной информации о выбранном проекте.
styles.css	Стили карточек, header, кнопок и состояний hover.
projects.js	Mock-коллекция проектов и операции добавления/удаления.
app.js	Фильтрация, рендер карточек и переход на страницу подробнее.
package.json	Описание npm-проекта и сценариев запуска.

2.4 Формирование карточек

Интерфейс формируется динамически. Функция renderProjects() очищает контейнер и создает карточку для каждого элемента коллекции. Такой подход позволяет применять одну и ту же разметку к любому числу объектов.

```
function renderProjects(items) {
  const list = document.querySelector('#projects');
  list.innerHTML = '';

  items.forEach(project => {
    const card = document.createElement('article');
```

```
  card.className = 'project-card';
  card.innerHTML = `<h3>${project.name}</h3><p>${project.language}</p>`;
  list.append(card);
 });
}
```

2.5 Фильтрация

Фильтрация выполняется по имени или языку проекта. При вводе текста формируется новый массив через filter(), после чего карточки перерисовываются. Исходная mock-коллекция не изменяется.

```
function filterProjects(query) {
 const q = query.toLowerCase();
 return projects.filter(project =>
  project.name.toLowerCase().includes(q) ||
  project.language.toLowerCase().includes(q)
 );
}
```

2.6 Добавление, удаление и переход на страницу подробнее

Кнопка добавления копирует первую карточку, присваивая новой записи новый идентификатор. Кнопка удаления удаляет проект из массива и запускает повторный рендер. На страницу подробнее передается идентификатор проекта через query-параметр. В header обеих страниц размещается ссылка «Домой», возвращающая на index.html.

```
function addProject() {
 const first = projects[0];
 const nextId = Math.max(...projects.map(p => p.id)) + 1;
 projects.push({ ...first, id: nextId, name: first.name + ' copy' });
 renderProjects(projects);
}

function deleteProject(id) {
 const index = projects.findIndex(p => p.id === id);
 if (index !== -1) projects.splice(index, 1);
 renderProjects(projects);
}
```

2.7 Роль Node.js и npm

На этапе ЛР3 Node.js используется как среда запуска инструментов разработки, а npm - как менеджер пакетов и команд проекта. Сам интерфейс

пока работает с mock-данными в браузере. Такой этап соответствует переходу от полностью статической страницы к проекту, имеющему собственную структуру, package.json и сценарии запуска.

2.8 Порядок демонстрации лабораторной работы № 3

Запустить frontend и показать страницу списка проектов.

Показать mock-коллекцию и объяснить поля объекта project.

Ввести строку фильтра и показать изменение набора карточек.

Нажать «Добавить» и показать копирование первой карточки.

Удалить карточку и показать изменение коллекции.

Открыть страницу подробнее и вернуть пользователя кнопкой «Домой».

В DevTools открыть Network, перезагрузить страницу и показать GET-запрос HTML и содержимое Response.

2.9 Ответы на контрольные вопросы

2.9.1 Структура проекта

Проект разделен на HTML-страницы, CSS, JavaScript-модули и package.json. HTML задает структуру, CSS - представление, JavaScript - данные и интерактивность, package.json - метаданные npm-проекта и команды запуска. Разделение облегчает поддержку и позволяет далее заменить mock-данные на серверный API без полной переработки интерфейса.

2.9.2 Что такое MVC

MVC - архитектурный шаблон Model-View-Controller. Model отвечает за данные и бизнес-логику, View - за отображение, Controller - за обработку действий пользователя и координацию модели и представления. В учебной ЛР3

массив projects играет роль модели, DOM-разметка карточек - представления, а функции фильтрации, добавления и удаления - упрощенного контроллера.

2.9.3 Что такое роутинг

Роутинг - сопоставление адреса или маршрута с определенным обработчиком или представлением. На клиентской стороне это может быть переход между index.html и project.html. На серверной стороне, начиная с ЛР4, маршруты вида GET /api/projects или DELETE /api/projects/:id будут сопоставляться с обработчиками API.

2.9.4 Ключевые этапы рендера страницы

Браузер получает HTML, строит DOM, загружает и разбирает CSS в CSSOM, формирует render tree, вычисляет размеры и положение элементов на этапе layout, затем выполняет paint и compositing. JavaScript может изменять DOM и стили, после чего часть этапов выполняется повторно.

3 Лабораторная работа № 4. REST API на Node.js

3.1 Постановка задачи

Для ЛР4 необходимо реализовать собственный web-сервис API на Node.js, хранить данные в JSON-файле и проверить через Postman или Insomnia пять типов операций: список с фильтрацией, получение одной записи, добавление, редактирование и удаление [1].

3.2 Переход от mock-данных к серверу

На этом этапе массив projects перестает быть единственным источником данных. Метаданные анализируемых проектов сохраняются в projects.json, а доступ к ним предоставляет Node.js-сервер. Это учебная реализация, созданная специально под требования ЛР4; production Local API CodeSlicer реализован отдельно и имеет другую внутреннюю архитектуру.

```
[
 {
  "id": 1,
  "name": "FastAPI",
  "language": "Python",
  "files": 3099,
  "status": "Analyzed"
 }
]
```

3.3 API-маршруты

Метод	URL	Назначение	Основные коды
GET	/api/projects	Получить список; поддерживает фильтрацию через query	200
GET	/api/projects/:id	Получить один проект	200, 404
POST	/api/projects	Добавить проект	201, 400
PUT	/api/projects/:id	Изменить проект	200, 400, 404
DELETE	/api/projects/:id	Удалить проект	204, 404

3.4 Пример router

Маршруты удобно объединить в отдельном Express Router. Обработчики получают данные, проверяют параметры, выполняют операцию над коллекцией и возвращают JSON либо соответствующий HTTP status code.

```
const express = require('express');
const router = express.Router();

router.get('/projects', getProjects);
router.get('/projects/:id', getProjectById);
router.post('/projects', createProject);
router.put('/projects/:id', updateProject);
router.delete('/projects/:id', deleteProject);

module.exports = router;
```

3.5 Добавление нового элемента

При POST-запросе сервер проверяет обязательные поля, формирует новый идентификатор, добавляет объект в массив и сохраняет обновленную коллекцию в projects.json.

```
function createProject(req, res) {
 const project = req.body;
 if (!project.name || !project.language) {
  return res.status(400).json({ error: 'name and language are required' });
 }

 project.id = getNextId();
 projects.push(project);
 saveProjects();
 return res.status(201).json(project);
}
```

3.6 Фильтрация списка

Для GET /api/projects используется query-параметр q. Сервер возвращает только записи, в названии или языке которых встречается искомая строка. Фильтрация переносится с mock-уровня на backend, что подготавливает приложение к AJAX-взаимодействию в ЛР5.

```
GET /api/projects?q=python

200 OK
```

```
[
 { "id": 1, "name": "FastAPI", "language": "Python" }
]
```

3.7 Проверка в Postman

Для защиты создается коллекция запросов Postman. Сначала выполняется GET списка, затем POST новой записи, GET по ее id, фильтрация списка и DELETE. После удаления повторный GET должен подтверждать отсутствие записи. Для каждого запроса необходимо показать URL, метод, JSON body при его наличии, Response и status code.

3.8 Порядок демонстрации лабораторной работы № 4

Запустить Node.js API.

В Postman выполнить GET /api/projects и показать JSON-массив.

Добавить новый проект через POST и показать ответ 201 Created.

Получить созданную запись по id.

Выполнить фильтрацию списка через query-параметр.

Удалить запись и показать 204 No Content.

Повторить GET и подтвердить изменение projects.json.

3.9 Ответы на контрольные вопросы

3.9.1 Что такое Node.js

Node.js - среда выполнения JavaScript вне браузера, построенная вокруг событийной модели и неблокирующего ввода-вывода. Она подходит для web-серверов, API, CLI-инструментов и задач, где приложение обслуживает большое число операций ввода-вывода. npm используется для управления зависимостями и сценариями проекта.

3.9.2 Клиент-серверная модель. SSR и SPA

В клиент-серверной модели клиент отправляет запрос, сервер обрабатывает его и возвращает ресурс или данные. При SSR сервер формирует готовый HTML для каждого запроса. В SPA браузер получает базовую страницу и JavaScript, а дальнейшее состояние интерфейса меняется на клиенте, обычно через API. Учебный интерфейс CodeSlicer ближе к SPA: карточки формируются в браузере, а сервер предоставляет JSON.

3.9.3 Встроенные библиотеки Node.js

- http - создание HTTP-сервера без внешнего фреймворка;
- fs - чтение и запись files, в том числе projects.json;
- path - безопасная работа с файловыми путями;
- url - разбор URL и query-параметров;
- events - событийная модель и EventEmitter.

3.9.4 Методы API, JSON и коды состояний

GET /api/projects возвращает массив проектов и 200. GET /api/projects/:id возвращает один объект или 404. POST /api/projects принимает JSON нового проекта и возвращает созданный объект с 201; при неверных данных - 400. PUT обновляет объект и возвращает 200 либо 404. DELETE при успешном удалении возвращает 204, а при неизвестном id - 404.

4 Лабораторная работа № 5. AJAX, XMLHttpRequest и CORS

4.1 Постановка задачи

ЛР5 продолжает клиентскую часть ЛР3. Теперь данные должны загружаться с API, созданного в ЛР4. Методические указания требуют использовать AJAX-запросы через XMLHttpRequest, запускать frontend и backend на разных сокетах и продемонстрировать влияние CORS. Также создается страница добавления/редактирования с редактируемыми полями, но кнопка сохранения появляется только в ЛР6 [1].

4.2 Замена mock-коллекции на XMLHttpRequest

Вместо прямого обращения к массиву интерфейс отправляет GET-запрос к API. После получения ответа JSON преобразуется в JavaScript-объекты и передается в существующую функцию renderProjects(). Поэтому визуальная часть ЛР3 сохраняется, а источник данных меняется.

```
function loadProjects(query = '') {
 const xhr = new XMLHttpRequest();
 xhr.open('GET', 'http://localhost:3001/api/projects?q=' + encodeURIComponent(query));

 xhr.onload = function () {
  if (xhr.status === 200) {
   const projects = JSON.parse(xhr.responseText);
   renderProjects(projects);
  }
 };

 xhr.send();
}
```

4.3 Проблема CORS

Если frontend открыт, например, с http://localhost:3000, а API работает на http://localhost:3001, origins различаются из-за номера порта. Браузер применяет Same-Origin Policy и не предоставляет JavaScript доступ к ответу, если сервер не разрешил такой cross-origin запрос соответствующими CORS-заголовками.

В соответствии с порядком показа лабораторной сначала демонстрируется ошибка в Network/Console. Затем включается CORS Unblock и запрос повторяется. В реальной разработке правильным решением является настройка CORS на сервере или размещение frontend и API на одном origin; расширение браузера используется только как учебный способ увидеть сам механизм.

4.4 Фильтрация через API

Поле фильтра больше не фильтрует локальный массив. При каждом подтвержденном поиске frontend формирует XHR к GET /api/projects?q=..., а сервер возвращает уже отфильтрованный набор. Во вкладке Network можно увидеть URL, query-параметр, Request Headers, Response Headers и JSON Response.

4.5 Страница добавления/редактирования

Страница редактирования получает существующий проект по id и заполняет поля формы. Пользователь может изменить значения локально, но в ЛР5 изменения не отправляются на сервер: кнопка «Сохранить» отсутствует. Такое ограничение специально соответствует методическим указаниям и позволяет вынести реальную запись данных в ЛР6.

```
function loadProject(id) {
 const xhr = new XMLHttpRequest();
 xhr.open('GET', 'http://localhost:3001/api/projects/' + id);
 xhr.onload = function () {
  if (xhr.status === 200) fillForm(JSON.parse(xhr.responseText));
 };
 xhr.send();
}
```

4.6 Порядок демонстрации лабораторной работы № 5

Запустить frontend и API на разных портах.

Открыть Network, выбрать фильтр XHR и выполнить поиск.

Показать CORS-ошибку при отключенном расширении.

Включить CORS Unblock и повторить тот же запрос.

Показать query-параметр и JSON-ответ в Network.

Через Postman добавить новый проект в API.

Обновить frontend и показать новую карточку.

Открыть страницу редактирования и показать заполненные поля без сохранения.

4.7 Ответы на контрольные вопросы

4.7.1 Что такое CORS и как решается проблема

CORS (Cross-Origin Resource Sharing) - механизм, с помощью которого сервер сообщает браузеру, какие внешние origins имеют право читать его ответы. Проблему можно решить настройкой Access-Control-Allow-Origin и других CORS-заголовков на API, reverse proxy, размещением frontend и API на одном origin либо, исключительно для локальной учебной демонстрации, временным browser extension. Отключать защиту браузера для production нельзя.

4.7.2 Что такое AJAX

AJAX - подход к обмену данными между браузером и сервером без полной перезагрузки страницы. Название исторически расшифровывается как Asynchronous JavaScript and XML, но современные приложения обычно передают JSON. AJAX описывает подход, а конкретным механизмом может быть XMLHttpRequest или Fetch API.

4.7.3 Что такое XMLHttpRequest

XMLHttpRequest - браузерный API для выполнения HTTP-запросов из JavaScript. Объект XHR позволяет задать метод и URL через open(), отправить запрос через send(), получить status и responseText, а также обработать события

load, error и progress. В ЛР5 он используется намеренно, чтобы затем в ЛР6 показать переход к Promise-based Fetch API.

5 Лабораторная работа № 6. Fetch API, промисы и сборка frontend

5.1 Постановка задачи

В ЛР6 необходимо заменить callback-подход XMLHttpRequest на промисы и Fetch API, использовать async/await, собрать frontend bundler-ом и развернуть bundle на сервере вместе с API. После этого AJAX-запросы выполняются к тому же origin, с которого загружена страница, поэтому CORS-расширение больше не требуется [1].

5.2 Переход с XMLHttpRequest на fetch

Функция загрузки проектов становится короче: fetch() возвращает Promise, response.json() также возвращает Promise, а async/await позволяет записать последовательность асинхронных операций в линейном виде.

```
async function loadProjects(query = '') {
 const response = await fetch('/api/projects?q=' + encodeURIComponent(query));

 if (!response.ok) {
  throw new Error('HTTP ' + response.status);
 }

 const projects = await response.json();
 renderProjects(projects);
}
```

5.3 Сохранение формы

В ЛР6 на странице добавления/редактирования появляется кнопка «Сохранить». Если у проекта уже есть id, выполняется PUT; для нового проекта - POST. Запрос отправляет JSON и после успешного ответа возвращает пользователя к списку.

```
async function saveProject(project) {
 const hasId = Boolean(project.id);
 const url = hasId ? `/api/projects/${project.id}` : '/api/projects';
 const method = hasId ? 'PUT' : 'POST';

 const response = await fetch(url, {
  method,
  headers: { 'Content-Type': 'application/json' },
```

```
  body: JSON.stringify(project)
 });

 if (!response.ok) throw new Error('Save failed');
 return response.json();
}
```

5.4 Промисы и async/await

Promise представляет результат асинхронной операции, который может находиться в состоянии pending, fulfilled или rejected. Методы then(), catch() и finally() позволяют подписаться на завершение. Ключевое слово await приостанавливает выполнение текущей async-функции до завершения промиса, но не блокирует весь JavaScript runtime.

5.5 Сборка frontend

Исходные JavaScript-файлы объединяются bundler-ом. Для учебного проекта можно использовать esbuild: он требует минимальной конфигурации и позволяет собрать app.js и импортируемые модули в один bundle.js. В ветке frontend остаются исходники, а собранный bundle переносится в серверную часть ЛР4, как требует методика.

```
// package.json
{
 "scripts": {
  "build": "esbuild src/app.js --bundle --outfile=dist/bundle.js"
 }
}
```

5.6 Размещение frontend и API на одном сервере

На финальном этапе Node.js-сервер раздает index.html и bundle.js как статические файлы и одновременно обслуживает /api/projects. Браузер получает страницу и выполняет fetch по относительному URL /api/projects, поэтому схема становится same-origin.

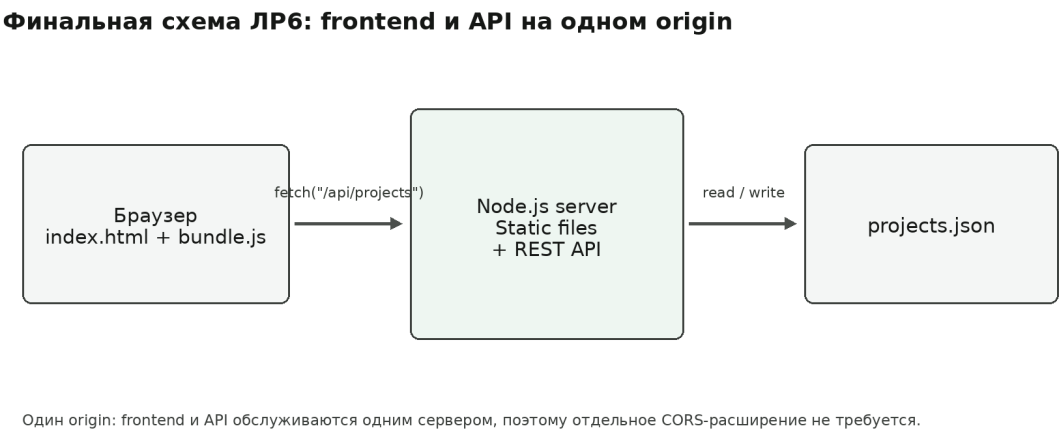


Рисунок 3 - Same-origin схема итоговой версии ЛР6

5.7 Связь с production CodeSlicer

Именно на этапе ЛР6 учебная архитектура становится наиболее похожей на текущий Local Hub CodeSlicer. В production-клиенте запросы также выполняются через fetch по относительным путям, а frontend и локальный API работают совместно. Различие состоит в том, что настоящий CodeSlicer использует собственный Python local API и значительно более сложную предметную модель, тогда как учебный модуль использует Node.js и projects.json только для выполнения требований курса [2].

Репозиторий также содержит отдельное VS Code extension на TypeScript: в extensions/vscode/package.json присутствуют tsc-скрипты compile/watch и TypeScript в devDependencies. Это хороший пример того, что bundling/compilation и TypeScript действительно являются частью реальной экосистемы проекта, хотя учебная ЛР6 может быть реализована проще [2].

5.8 Порядок демонстрации лабораторной работы № 6

Запустить сервер, который одновременно раздает frontend bundle и API.

В DevTools показать, что в Sources используется собранный bundle, а не отдельные исходные JS-файлы.

Выполнить фильтрацию и показать fetch-запрос в Network.

Показать, что CORS-расширение отключено и запрос проходит, потому что origin совпадает.

Открыть форму, изменить проект и сохранить его через PUT.

Добавить новую запись через POST и удалить существующую через DELETE.

Показать async/await и обработку response.ok в исходном коде.

5.9 Ответы на контрольные вопросы

5.9.1 Что такое Promise и fetch

Promise - объект, представляющий будущий результат асинхронной операции. Fetch API использует промисы для HTTP-запросов: fetch(url) возвращает Promise<Response>. Сам факт получения HTTP-ответа завершает fetch успешно даже при коде 404 или 500, поэтому приложение должно отдельно проверять response.ok или response.status.

5.9.2 Что такое async/await

async объявляет функцию, результат которой всегда оборачивается в Promise. await можно использовать внутри async-функции для ожидания другого Promise. Конструкция делает асинхронный код последовательным по форме, а ошибки можно обрабатывать обычным try/catch.

5.9.3 Что такое bundler

Bundler анализирует граф импортов frontend-проекта и формирует один или несколько файлов для публикации. Он может объединять модули,

выполнять tree shaking, минификацию и преобразования. Примеры: esbuild, Vite/Rollup и Webpack.

5.9.4 Что такое Babel

Babel - транспайлер JavaScript. Он преобразует современный синтаксис в форму, совместимую с целевыми браузерами, и при необходимости подключает плагины и presets. Babel не является обязательным bundler-ом: часто он используется как отдельный этап внутри системы сборки.

5.9.5 Что такое TypeScript

TypeScript - надмножество JavaScript со статической системой типов. Код TypeScript компилируется в JavaScript. Типизация помогает находить ошибки до выполнения и упрощает поддержку крупных проектов. В CodeSlicer TypeScript используется, в частности, в исходниках VS Code extension, а сборка extension выполняется через tsc [2].

6 Связь лабораторных работ № 3-6

Все четыре лабораторные работы используют одну предметную сущность - проект, который может быть передан CodeSlicer для анализа. Меняется не смысл данных, а архитектурный способ работы с ними. Благодаря этому отчет представляет не четыре несвязанных задания, а последовательную эволюцию одного web-приложения.

ЛР	**Источник данных**	**Клиент-серверное взаимодействие**	**Новый основной навык**
3	Mock-массив JavaScript	Нет отдельного API	Компоненты, коллекции, фильтрация, динамический DOM
4	projects.json	REST API Node.js	HTTP-методы, маршруты, JSON, status codes
5	projects.json через API	XMLHttpRequest, разные origins	AJAX, Network, CORS
6	projects.json через API	fetch, async/await, same-origin	Promise, Fetch API, bundler, deployment

Такой путь соответствует общей логике дисциплины: сначала данные существуют непосредственно в клиентском коде, затем появляются сервер и API, после чего frontend подключается к backend сначала традиционным XHR, а затем современным fetch. На финальном этапе интерфейс и API разворачиваются совместно.

Заключение

В отчете рассмотрен CodeSlicer как реальный проект и как предметная область для лабораторных работ № 3-6. CodeSlicer решает задачу анализа влияния изменений в программной системе: строит граф проекта, сохраняет сведения о происхождении связей и предоставляет результаты через CLI, Local Hub, VS Code и AI-интеграции.

Для выполнения требований курса выделен отдельный учебный web-модуль управления анализируемыми проектами. В лабораторной работе № 3 он работает с mock-коллекцией и динамическими карточками. В лабораторной работе № 4 данные переносятся в JSON-файл и предоставляются через REST API Node.js. В лабораторной работе № 5 frontend подключается к API через XMLHttpRequest, что позволяет изучить AJAX и CORS. В лабораторной работе № 6 запросы переводятся на Fetch API и async/await, frontend собирается bundler-ом и размещается вместе с API на одном origin.

Такой подход сохраняет связь с производственным CodeSlicer, но не подменяет его реальную архитектуру учебной. Наиболее близким к production-схеме становится итог ЛР6: текущий Local Hub CodeSlicer также использует same-origin Fetch API и отделяет web-интерфейс от ядра анализа. В совокупности лабораторные работы демонстрируют последовательный переход от простого browser-side приложения к полноценной client-server архитектуре.

Список использованных источников

1. Репозиторий курса «Проектирование сетевых приложений». Требования к лабораторным работам № 3-6. URL: https://github.com/iu5git/JavaScript

2. CodeSlicer. Репозиторий проекта и документация. URL: https://github.com/artemnoor/CodeSlicer

3. «Ваш AI-агент не понимает код. Он просто очень уверенно угадывает - поэтому мы создали SLICER». Хабр. URL: https://habr.com/ru/articles/1063004/

4. CodeSlicer Demo. URL: https://code-slicer.ru/demo/

5. MDN Web Docs. XMLHttpRequest, Fetch API, Promise, CORS. URL: https://developer.mozilla.org/

6. Node.js Documentation. URL: https://nodejs.org/docs/